CO1 – Assessment Tool 1: IT Application-Based Problem Solving

Scalability Audit of "Shop Sphere" E-Commerce Platform

Name: M Vishnu Vardhan Reddy

Reg. No.: 192572062

Course Code: CSA0601

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

A. **(a) Understand the Problem**

The input size for all modules is mainly represented by **n**, where n is the number of products, requests, or records being processed.

Module	**Input Size**	**Basic Operation**	**Where It Occurs**	**Expected Growth**
M1 – Duplicate Detector	n products	Product title and seller comparison	Inside the nested loops	$\Theta(n^2)$
M2 – API Rate Limiter	n requests	Audit(Request[j])	Inside the inner loop	$\Theta(n)$
M3 – Product Search S1	n products	Product-key comparison	During sequential scan	$\Theta(n)$ worst case
M3 – Product Search S2	n sorted products	Key comparison with middle element	At every recursive/iterative step	$\Theta(\log n)$
M4 – Recommendation Engine	n customer-product records	Merge/processing operation	At every level of recursion	Current: $\Theta(n^2)$, Proposed: $\Theta(n \log n)$
M5 – Sales Analytics	n sales records	Sorting/merge processing	At every recursive level	$\Theta(n \log^2 n)$

Explanation

M1: Every product is compared with almost every other product. Therefore, the number of comparisons grows quadratically.

M2: Although the outer loop executes logarithmically many times, the inner loop sizes are 1, 2, 4, 8, ... Their total is a geometric series, giving $\Theta(n)$, not $\Theta(n \log n)$.

M3: S1 may examine every product, while S2 removes half of the remaining search space after every comparison.

M4: The current design has four recursive subproblems of size n/2, producing $\Theta(n^2)$. Removing replication reduces it to two subproblems and gives $\Theta(n \log n)$.

M5: The recurrence has an n log n amount of work at every recursion level, resulting in $\Theta(n \log^2 n)$.

(b) Analyse the Iterative Modules

M1 – Duplicate-Listing Detector

The loops are:

```
FOR i = 0 TO n-2
    FOR j = i+1 TO n-1
        comparison
```

For each value of i:

- When i = 0, comparisons = n - 1
- When i = 1, comparisons = n - 2
- When i = 2, comparisons = n - 3
- ...
- When i = n - 2, comparisons = 1

Therefore,

$$C(n) = (n-1) + (n-2) + \cdots + 2 + 1$$

Using the sum of first n-1 integers:

$$C(n) = \frac{(n-1)n}{2}$$

Therefore,

$$\boxed{C(n) = \frac{n(n-1)}{2}}$$

Time Complexity

$$C(n) = \frac{n^2 - n}{2}$$

The highest-order term is n^2.

$$\boxed{T(n) = \Theta(n^2)}$$

Space Complexity

Only loop variables and temporary comparison information are required.

$$\boxed{S(n) = \Theta(1)}$$

assuming duplicate markings are stored as part of the existing product records rather than a new n-sized auxiliary structure.

Interpretation

M1 becomes extremely expensive as the catalogue grows because doubling the number of products approximately quadruples the work.

M2 – API Rate Limiter

The values of i are:

$$1,2,4,8,16, \ldots$$

The inner loop performs exactly i audit operations.

Therefore, the total is:

$$1 + 2 + 4 + 8 + \cdots$$

Suppose the largest value of i that is less than n is:

$$2^{k-1}$$

where

$$k = \lceil \log_2 n \rceil$$

Therefore,

$$A(n) = 1 + 2 + 4 + \cdots + 2^{k-1}$$

Using the geometric-series formula:

$$A(n) = 2^k - 1$$

Therefore,

$$\boxed{A(n) = 2^{\lceil \log_2 n \rceil} - 1}$$

Complexity

Since

$$n \leq 2^{\lceil \log_2 n \rceil} < 2n$$

we have:

$$A(n) = \Theta(n)$$

Therefore, the team member's claim:

$$\Theta(n \log n)$$

is incorrect.

The correct tight bound is:

$$\boxed{\Theta(n)}$$

Verification for n = 1,024

Since:

$$1024 = 2^{10}$$

the number of operations is:

$$A(1024) = 2^{10} - 1$$
$$\boxed{\mathrm{A}(1024) = 1023}$$

So M2 performs 1,023 Audit operations.

(c) Solve the Recurrence by Substitution

Product Search S2

Given:

$$\mathrm{T(n)} = \mathrm{T(n/2)} + 1$$

and

$$T(1) = 1$$

We solve using backward substitution.

Step 1

$$\mathrm{T(n)} = \mathrm{T(n/2)} + 1$$

Step 2

Substitute:

$$T(n/2) = T(n/4) + 1$$

Therefore,

$$T(n) = T(n/4) + 1 + 1$$
$$T(n) = T(n/4) + 2$$

Step 3

Again:

$$T(n/4) = T(n/8) + 1$$

Therefore,

$$T(n) = T(n/8) + 3$$

Step 4

Continuing:

$$T(n) = T(n/16) + 4$$

After k substitutions:

$$T(n) = T\left(\frac{n}{2^k}\right) + k$$

The recursion reaches the base case when:

$$\frac{n}{2^k} = 1$$

Therefore:

$$n = 2^k$$

Taking logarithm:

$$k = \log_2 n$$

Substitute:

$$T(n) = T(1) + \log_2 n$$

Since:

$$T(1) = 1$$

we get:

$$\boxed{T(n) = \log_2 n + 1}$$

for powers of two.

Thus,

$$\boxed{T(n) = \Theta(\log n)}$$

Space Complexity

Iterative S2

An iterative implementation only requires a few variables.

$$\boxed{S(n) = \Theta(1)}$$

Recursive S2

Each recursive call remains on the call stack.

The number of calls is:

$$\log_2 n + 1$$

Therefore:

$$S(n)=\Theta(\log n)$$

(d) Apply the Master Theorem

M4 – Current Design

Given:

$$T(n) = 4T(n/2) + n$$

Compare with:

$$T(n) = aT(n/b) + f(n)$$

Therefore:

$$a = 4$$
$$b = 2$$
$$f(n) = n$$

Calculate:

$$n^{\log_b a} = n^{\log_2 4}$$
$$= n^2$$

Therefore:

$$n^{\log_b a} = n^2$$

Compare:

$$f(n) = n$$

with:

$$n^2$$

We have:

$$n = O(n^{2-\epsilon})$$

for a suitable positive ε.

Therefore, **Master Theorem Case 1** applies.

Hence:

$$\boxed{T(n) = \Theta(n^2)}$$

Effect of doubling the data

If n becomes 2n:

$$T(2n) \approx (2n)^2$$
$$= 4n^2$$

Therefore, the running time becomes approximately **4 times larger**.

M4 – Proposed Redesign

Given:

$$T(n) = 2T(n/2) + n$$

Therefore:

$$a = 2$$
$$b = 2$$
$$f(n) = n$$

Calculate:

$$n^{\log_b a} = n^{\log_2 2}$$
$$= n$$

Thus:

$$f(n) = \Theta(n^{\log_b a})$$

Therefore, **Master Theorem Case 2** applies.

Hence:

$$\boxed{T(n) = \Theta(n \log n)}$$

Effect of doubling the data

For the dominant term:

$$T(n) = n\log_2 n$$

When data doubles:

$$T(2n) = 2n\log_2(2n)$$

$$= 2n(\log_2 n + 1)$$

So the running time increases by slightly more than **2 times**, rather than 4 times.

M4 Comparison

Design	**Recurrence**	**Complexity**	**Effect of Doubling**
Current	4T(n/2)+n	Θ(n²)	Approximately 4×
Proposed	2T(n/2)+n	Θ(n log n)	Approximately 2×

Therefore, the proposed design is significantly more scalable.

M5 – Sales Analytics Engine

Given:

$$T(n) = 2T(n/2) + n\log n$$

Compare with:

$$T(n) = aT(n/b) + f(n)$$

Therefore:

$$a = 2$$
$$b = 2$$
$$f(n) = n\log n$$

Calculate:

$$n^{\log_b a} = n^{\log_2 2} = n$$

Thus:

$$f(n) = n\log n$$

The standard Master Theorem cases do not directly give the result because the extra logarithmic factor occurs at the boundary.

Using the extended Master Theorem:

$$f(n) = \Theta(n^{\log_b a}\log^1 n)$$

Therefore:

$$\boxed{T(n) = \Theta(n\log^2 n)}$$

Recursion Tree Explanation

At the root:

$$n\log n$$

At the next level:

$$2\left(\frac{n}{2}\log\frac{n}{2}\right) = n(\log n - 1)$$

At the next level:

$$n(\log n - 2)$$

Therefore, the total internal work is:

$$n[\log n + (\log n - 1) + (\log n - 2) + \cdots + 1]$$

This contains approximately:

$$n(\log n)^2$$

Therefore:

$$T(n)=\Theta(n\log 2n)$$

(e) Best, Average and Worst Cases

M3 – Search S1

S1 performs sequential searching.

Best Case

The searched product is the **first product**.

$$\boxed{1 \text{ comparison}}$$

Average Case

If the product exists and is equally likely to occur at any position:

$$\frac{1 + 2 + \cdots + n}{n}$$
$$= \frac{\frac{n(n+1)}{2}}{n}$$
$$= \frac{n+1}{2}$$

Therefore:

$$\boxed{\frac{n+1}{2}}$$

comparisons.

Worst Case

The product is at the last position or does not exist.

$$\boxed{n \text{ comparisons}}$$

S1 Average Comparisons When Existence Probability is p

Probability that product exists:

$$p$$

Probability that product does not exist:

$$1 - p$$

If it exists, expected comparisons:

$$\frac{n+1}{2}$$

If it does not exist:

$$n$$

Therefore:

$$E[C] = p\,(\frac{n+1}{2}) + (1-p)n$$

Simplifying:

$$E[C] = \frac{pn+p}{2} + n - pn$$
$$E[C] = n - \frac{pn}{2} + \frac{p}{2}$$

Therefore:

$$\boxed{E[C] = n\,(1 - \frac{p}{2}) + \frac{p}{2}}$$

For n = 1,000,000 and p = 0.5

$$E[C] = 0.5\,(\frac{1{,}000{,}001}{2}) + 0.5(1{,}000{,}000)$$
$$= 250{,}000.25 + 500{,}000$$
$$\boxed{E[C] = 750{,}000.25}$$

So approximately:

$$\boxed{750{,}000 \text{ comparisons}}$$

are required on average.

S2 – Binary Search

Best Case

The searched product is exactly the middle element.

$$\boxed{1 \text{ comparison}}$$

Average Case

The search space is halved at each step.

$$\boxed{\Theta(\log n)}$$

comparisons.

Worst Case

The product is at the deepest level or is not present.

For the recurrence given:

$$T(n) = T(n/2) + 1$$

the comparison count for power-of-two n is:

$$\boxed{\log_2 n + 1}$$

Thus, the worst-case complexity is:

$$\boxed{\Theta(\log n)}$$

(f) Prove an Asymptotic Bound

Given:

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

We need to prove:

$$\boxed{f(n) = \Theta(n^2)}$$

For Θ notation, we need constants c_1, c_2, and n_0 such that:

$$0 \leq c_1 n^2 \leq f(n) \leq c_2 n^2$$

for all:

$$n \geq n_0$$

Lower Bound

Since all terms are non-negative:

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

Therefore:

$$f(n) \geq 3n^2$$

Choose:

$$\boxed{c_1 = 3}$$

Thus:

$$3n^2 \leq f(n)$$

Upper Bound

For:

$$n \geq 2$$

we have:

$$\log_2 n \leq n$$

Therefore:

$$10n\log_2 n \leq 10n^2$$

Also, for:

$$n \geq 1$$
$$50 \leq 50n^2$$

Hence:

$$f(n) \leq 3n^2 + 10n^2 + 50n^2$$
$$f(n) \leq 63n^2$$

Choose:

$$\boxed{c_2 = 63}$$

and:

$$\boxed{n_0 = 2}$$

Therefore:

$$3n^2 \leq f(n) \leq 63n^2$$

for all:

$$n \geq 2$$

Hence:

$$\boxed{f(n) = \Theta(n^2)}$$

Prove that f(n) is Not O(n)

Assume for contradiction that:

$$f(n) = O(n)$$

Then there must exist constants c > 0 and n_0 such that:

$$f(n) \leq cn$$

for all sufficiently large n.

But:

$$f(n) \geq 3n^2$$

Therefore:

$$\frac{f(n)}{n} \geq 3n$$

As:

$$n \to \infty$$
$$3n \to \infty$$

So no constant c can satisfy:

$$f(n) \leq cn$$

for all sufficiently large n.

Therefore:

$$\boxed{f(n) \notin O(n)}$$

Thus the report generator is quadratic, not linear.

(g) Scalability Decision

The server executes:

$$10^8$$

basic operations per second.

For:

$$n = 1{,}000{,}000$$

we estimate the operations and execution time.

M1

Exact comparisons:

$$\frac{n(n-1)}{2}$$
$$= \frac{1{,}000{,}000(999,\ 999)}{2}$$

$$\boxed{499{,}999{,}500{,}000}$$

operations.

Time:

$$\frac{499{,}999{,}500{,}000}{100{,}000{,}000} = 4{,}999.995 \text{ seconds}$$

Convert to minutes:

$$\frac{4{,}999.995}{60} \approx 83.33 \text{ minutes}$$

$$\boxed{\text{M1} \approx 83.33 \text{ minutes}}$$

Therefore, M1 **fails** the 10-minute nightly window.

M2

$$A(n) = 2^{\lceil \log_2 n \rceil} - 1$$

For:

$$n = 1{,}000{,}000$$
$$\lceil \log_2(1,000,000) \rceil = 20$$

Therefore:

$$A(n) = 2^{20} - 1$$
$$\boxed{1{,}048{,}575}$$

operations.

Time:

$$\frac{1{,}048{,}575}{10^8} = 0.01048575 \text{ seconds}$$

$$\boxed{\text{M2} \approx 0.0105 \text{ seconds}}$$

M2 easily meets the requirement.

S2 – Binary Search

For:

$$n = 1{,}000{,}000$$
$$\log_2(1,000,000) \approx 19.93$$

The maximum number of comparisons is approximately:

$$\boxed{20}$$

Time:

$$\frac{20}{10^8}$$
$$= 2 \times 10^{-7} \text{ seconds}$$
$$\boxed{\text{S2} \approx 0.0000002 \text{ seconds}}$$

Therefore, S2 is extremely scalable.

M4 – Current Design

Current recurrence:

$$T(n) = 4T(n/2) + n$$

Its complexity is:

$$\Theta(n^2)$$

For an estimate at n = 1,000,000, using the power-of-two form:

$$T(n) \approx 2n^2 - n$$

Therefore:

$$T(1,000,000) \approx 2(10^6)^2 - 10^6$$
$$\boxed{\approx 1{,}999{,}999{,}000{,}000}$$

operations.

Time:

$$\frac{1{,}999{,}999{,}000{,}000}{10^8}$$
$$\approx 19{,}999.99 \text{ seconds}$$
$$\approx 333.33 \text{ minutes}$$
$$\boxed{\approx 5.56 \text{ hours}}$$

Therefore, the current M4 design fails badly.

M4 – Proposed Design

Proposed recurrence:

$$T(n) = 2T(n/2) + n$$

For powers of two:

$$T(n) = n(\log_2 n + 1)$$

For n = 1,000,000:

$$T(n) \approx 1{,}000{,}000(19.93 + 1)$$
$$\boxed{\approx 20{,}931{,}569}$$

operations.

Time:

$$\frac{20{,}931{,}569}{10^8}$$
$$\approx 0.2093 \text{ seconds}$$
$$\boxed{\text{M4 proposed} \approx 0.21 \text{ seconds}}$$

Therefore, the proposed design easily meets the 10-minute requirement.

M5 – Sales Analytics Engine

M5 has:

$$T(n) = 2T(n/2) + n \log n$$

and:

$$\boxed{T(n) = \Theta(n\log^2 n)}$$

For:

$$n = 1{,}000{,}000$$
$$\log_2 n \approx 19.93$$

An estimate of the dominant work is:

$$n(\log_2 n)^2$$
$$= 1{,}000{,}000(19.93)^2$$
$$\approx 397{,}267{,}000$$

basic operations.

Therefore, the estimated time is:

$$\frac{397{,}267{,}000}{10^8}$$
$$\approx 3.97 \text{ seconds}$$
$$\boxed{\text{M5} \approx 4 \text{ seconds}}$$

Thus, M5's recursive analytics algorithm is within the 10-minute window.

M5 Report Generator

The separately profiled report generator has:

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

For:

$$n = 1{,}000{,}000$$
$$3n^2 = 3{,}000{,}000{,}000{,}000$$

and:

$$10n\log_2 n \approx 199{,}315{,}686$$

Therefore:

$$f(n) \approx 3{,}000{,}199{,}315{,}736$$

operations.

Time:

$$\frac{3{,}000{,}199{,}315{,}736}{10^8}$$
$$\approx 30{,}001.99 \text{ seconds}$$
$$\approx 500.03 \text{ minutes}$$
$$\boxed{\approx 8.33 \text{ hours}}$$

Therefore, the nightly report generator must also be redesigned if it is part of the nightly processing window.

Final Scalability Comparison

Module	**Approx. Operations for n = 1,000,000**	**Approx. Time**	**10-Minute Window**	**Decision**
M1 Duplicate Detector	499,999,500,000	83.33 min	Fails	Redesign
M2 Rate Limiter	1,048,575	0.0105 sec	Pass	Keep
S2 Binary Search	≈20	≈0.0000002 sec	Pass	Keep
M4 Current	≈1,999,999,000,000	333.33 min	Fails	Redesign
M4 Proposed	≈20,931,569	≈0.21 sec	Pass	Use proposed design
M5 Analytics	≈397 million*	≈4 sec*	Pass	Keep/optimize if needed
M5 Report Generator	≈3.000 trillion	≈500 min	Fails	Redesign

*M5's exact operation count depends on the base-case convention and non-power-of-two handling; the table gives a dominant-term estimate suitable for scalability analysis.

Final Conclusion

Shop Sphere's major scalability problems are caused by quadratic algorithms. M1 and the current M4 design become impractical when the catalogue grows to one million products. The proposed M4 redesign changes the growth from quadratic to n log n, while a hash-based M1 can reduce duplicate detection to expected linear time. M2 and S2 are already highly scalable. The separate M5 report generator is also a critical bottleneck because its $\Theta(n^2)$ cost exceeds the 10-minute nightly window. Therefore, redesigning the quadratic components before the catalogue expansion is essential for maintaining reliable platform performance.